P1135R4
The C++20 Synchronization Library

Published Proposal, 2019-03-04

Authors:

Bryce Adelstein Lelbach (NVIDIA)

Olivier Giroux (NVIDIA)

JF Bastien (Apple)

Detlef Vollmann

David Olsen (NVIDIA)

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LWG

Table of Contents

§ 1. Introduction

This paper is the unification of a series of related C++20 proposals for introducing new synchronization and thread coordination facilities and enhancing existing ones:

— [P0514R4]: Efficient `atomic` waiting and semaphores.

— [P0666R2]: Latches and barriers.

— [P0995R1]: `atomic_flag::test` and lockfree integral types.

— [P1258R0]: Don't make C++ unimplementable for small CPUs.

§ 2. Changelog

Revision 0: Post Rapperswil 2018 changes from [P0514R4], [P0666R2], and [P0995R1] based on Rapperswil 2018 LEWG feedback.

— Refactored `basic_barrier` and `barrier` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.

— Refactored `basic_semaphore` and `counting_semaphore` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.

— Fixed `update` parameters in semaphore, latch, and barrier member functions to consistently default to 1 to resolve mistakes identified by LEWG at Rapperswil 2018.

Revision 1: Pre San Diego 2018 changes based on Rapperswil 2018 LEWG feedback and a June discussion on the LEWG and SG1 mailing lists.

— Added member function versions of `atomic_wait_*` and `atomic_notify_*`, for consistency. Refactored wording to accommodate this.

— Renamed the `atomic_flag` overloads of `atomic_wait` and `atomic_wait_explicit` to `atomic_flag_wait` and `atomic_flag_wait_explicit` for consistency and to leave the door open for future compatibility with C.

— Renamed `latch::arrive_and_wait` and `barrier::arrive_and_wait` to `latch::sync` and `barrier::sync`, because LEWG at Rapperswil 2018 expected these methods to be the common use case and prefers they have a short name.

— Renamed `latch::arrive` to `latch::count_down` to further separate and distinguish the `latch` and `barrier` interfaces.

— Removed `barrier::try_wait` to resolve concerns raised during LEWG discussion at Rapperswil 2018 regarding its "maybe consuming" nature.

— Required that `barrier::arrival_token`'s move constructor and move assignment operators are `noexcept` to resolve discussions in LEWG at Rapperswil 2018 regarding exceptions being thrown when using the split arrive and wait barrier interface.

— Made `counting_semaphore::acquire`, `counting_semaphore::try_acquire`, and `latch::wait` `noexcept`, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.

— Made `counting_semaphore`, `latch`, and `barrier`'s constructors non `constexpr` and allowed them to throw `system_error` if the object cannot be created, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.

— Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::sync`, `barrier::wait`, `barrier::sync`, and `barrier::arrive_and_drop` throw nothing (but cannot be `noexcept`, because they have preconditions) to resolve discussions in LEWG at Rapperswil 2018 and on the mailing list.

Revision 2: San Diego 2018 changes to incorporate [P1258R0] and pre-meeting feedback.

— Made `barrier::wait` take its `arrival_token` parameter by rvalue reference.

— Made the `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types optional for freestanding implementations, as per [P1258R0].

Revision 3: Pre Kona 2019 changes based on San Diego 2018 LEWG feedback.

— Renamed `latch::sync` and `barrier::sync` back to `latch::arrive_and_wait` and `barrier::arrive_and_wait`, because this name had the strongest consensus in LEWG at San Diego 2018.

— Removed `atomic_int_fast_wait_t` and `atomic_uint_fast_wait_t`, because LEWG at San Diego 2018 felt that the use case was uncommon and the types had high potential for misuse.

— Made `counting_semaphore::acquire` and `latch::wait` non `noexcept` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.

— Made `counting_semaphore`, `latch`, and `barrier`'s constructors `constexpr` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to

allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.

— Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::arrive_and_wait`, `barrier::wait`, `barrier::arrive_and_wait`, and `barrier::arrive_and_drop` may throw `system_error` exceptions, which is an implication of the constructors of said objects being `constexpr` because any underlying system errors must be reported on operations not during construction.

— Added missing `atomic<T>::wait` and `atomic<T>::notify_*` member functions to the class synopses for the `atomic<T>` integral, floating-point, and pointer specializations.

— Fixed `atomic<T>::notify_*` member functions to be non `const`.

Revision 4: Lots of wording changes based on Kona 2019 LWG feedback. Three design changes to fix bugs that were discovered during LWG review or afterwards while revising the paper. These will be presented to SG1 in a separate paper (yet to be written) in Cologne.

— Changed `atomic_flag::test` to be a `const` function. Changed the `atomic_flag*` parameter of `atomic_flag_test` and `atomic_flag_test_explicit` to be `const atomic_flag*`.

— Added the requirement that the `least_max_value` template parameter to `counting_iterator` be greater than zero.

— Changed the requirement on the `update` parameter to `barrier::arrive` from `update >= 0` to `update > 0`.

§ 3. Wording

> Note: The following changes are relative to the post San Diego 2018 working draft of ISO/IEC 14882, ([N4791]).

> Note: The ❡ character is used to denote a placeholder number which shall be selected by the editor.

Add `<semaphore>`, `<latch>`, and `<barrier>` to Table 19 "C++ library headers" in **[headers]**.

Modify the header synopsis for `<atomic>` in **[atomics.syn]** as follows:

30.2 Header <atomic> synopsis **[atomics.syn]**

```
namespace std {
  // ...

  // 30.8, non-member functions
  // ...
  template<class T>
    void atomic_wait(const volatile atomic<T>*,
                     typename atomic<T>::value_type);
  template<class T>
    void atomic_wait(const atomic<T>*,
                     typename atomic<T>::value_type);
  template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_wait_explicit(const atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_notify_one(volatile atomic<T>*);
  template<class T>
    void atomic_notify_one(atomic<T>*);
  template<class T>
    void atomic_notify_all(volatile atomic<T>*);
  template<class T>
    void atomic_notify_all(atomic<T>*);

  // 30.3, type aliases
  // ...

  using atomic_intptr_t       = atomic<intptr_t>;
  using atomic_uintptr_t      = atomic<uintptr_t>;
  using atomic_size_t         = atomic<size_t>;
  using atomic_ptrdiff_t      = atomic<ptrdiff_t>;
  using atomic_intmax_t       = atomic<intmax_t>;
  using atomic_uintmax_t      = atomic<uintmax_t>;

  using atomic_signed_lock_free   = see below;
  using atomic_unsigned_lock_free = see below;
```

```
  // ...

  // 30.9, flag type and operations
  struct atomic_flag;
  bool atomic_flag_test(const volatile atomic_flag*) noexcept;
  bool atomic_flag_test(const atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_orde
  bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcep
  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_orc
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexc
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noe
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
  void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
  void atomic_flag_wait(const atomic_flag*, bool) noexcept;
  void atomic_flag_wait_explicit(const volatile atomic_flag*, bool, memory
  void atomic_flag_wait_explicit(const atomic_flag*, bool, memory_order) n
  void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
  void atomic_flag_notify_one(atomic_flag*) noexcept;
  void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;
  void atomic_flag_notify_all(atomic_flag*) const noexcept;
  #define ATOMIC_FLAG_INIT see below

  // 30.10, fences
  extern "C" void atomic_thread_fence(memory_order) noexcept;
  extern "C" void atomic_signal_fence(memory_order) noexcept;
}
```

Modify [**atomics.alias**] as follows:

30.3 Type aliases **[atomics.alias]**

1 The type aliases `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` are defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.

2 The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, other than `bool`. In freestanding implementations (4.1), these aliases are optional. Only implementations that provide an integral specialization of `atomic` other than `bool` for which `is_always_lock_free` is true, also provide `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. `is_always_lock_free` is `true` for `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. [*Note:* An implementation which provides these type aliases should choose the integral specialization of `atomic` for which the atomic waiting and notifying operations are most efficient. - *end note*]

Note: The reference to "atomic waiting and notifying operations" in the above change should refer to the new **[atomics.wait]** subclause.

Add a new subclause after **[atomics.lockfree]**:

30.  Waiting and notifying **[atomics.wait]**

1 ***Atomic waiting and notifying operations*** provide a mechanism to wait for the value of an atomic object to change more efficiently than can be achieved with polling.

2 The following functions are ***atomic waiting operations***:

(2.1) — `atomic<T>::wait`.

(2.2) — `atomic_flag::wait`.

(2.3) — `atomic_wait` and `atomic_wait_explicit`.

(2.4) — `atomic_flag_wait` and `atomic_flag_wait_explicit`.

3 The following functions are ***atomic notifying operations***:

(3.1) — `atomic<T>::notify_one` and `atomic<T>::notify_all`.

(3.2) — `atomic_flag::notify_one` and `atomic_flag::notify_all`.

(3.3) — `atomic_notify_one` and `atomic_notify_all`.

(3.4) — `atomic_flag_notify_one` and `atomic_flag_notify_all`.

4 Atomic waiting operations may block until they are unblocked by atomic notifying operations, according to each function's effects. [*Note:* Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *end note*]

Modify [**atomics.types.generic**] as follows:

30.7 Class template `atomic` **[atomics.types.generic]**

```
namespace std {
  template<class T> struct atomic {
    using value_type = T;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T, memory_order = memory_order::seq_cst) volatile noexcept;
    void store(T, memory_order = memory_order::seq_cst) noexcept;
    T load(memory_order = memory_order::seq_cst) const volatile noexcept;
    T load(memory_order = memory_order::seq_cst) const noexcept;
    operator T() const volatile noexcept;
    operator T() const noexcept;
    T exchange(T, memory_order = memory_order::seq_cst) volatile noexcept;
    T exchange(T, memory_order = memory_order::seq_cst) noexcept;
    bool compare_exchange_weak(T&, T, memory_order, memory_order) volatile
    bool compare_exchange_weak(T&, T, memory_order, memory_order) noexcept
    bool compare_exchange_strong(T&, T, memory_order, memory_order) volati
    bool compare_exchange_strong(T&, T, memory_order, memory_order) noexce
    bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst
    bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst
    bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_c
    bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_c
    void wait(T, memory_order = memory_order::seq_cst) const volatile noex
    void wait(T, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
    atomic() noexcept = default;
    constexpr atomic(T) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T operator=(T) volatile noexcept;
    T operator=(T) noexcept;
  };
}
```

Add the following to the end of [**atomics.types.operations**]:

```
void wait(T old, memory_order order = memory_order::seq_cst) const volatil
void wait(T old, memory_order order = memory_order::seq_cst) const noexcep
```

1 *Expects:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.

2 *Effects:* Repeatedly performs the following steps, in order:

(2.1) — Evaluates `load(order)` and compares its value representation for equality against that of `old`.

(2.2) — If they compare unequal, returns.

(2.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

3 *Remarks:* This function is an atomic waiting operation.

```
void notify_one() volatile noexcept;
void notify_one() noexcept;
```

4 *Effects:* Given the set *W* of atomic waiting operations such that:

(4.1) — each atomic waiting operation has blocked after observing the result of some atomic operation `X`,

(4.2) — `X` precedes some atomic operation `Y` in the modification order of `*this`, and

(4.3) — `Y` happens before this call.

If the set *W* is not empty, unblocks the execution of one operation in *W*.

5 *Remarks:* This function is an atomic notifying operation.

```
void notify_all() volatile noexcept;
void notify_all() noexcept;
```

6 *Effects:* Unblocks the execution of all atomic waiting operations such that:

(6.1) — each atomic waiting operation has blocked after observing the result of some atomic operation `X`,

(6.2) — `X` precedes some atomic operation `Y` in the modification order of `*this`, and

(6.3) — `Y` happens before this call.

7 *Remarks:* This function is an atomic notifying operation.

Modify [**atomics.types.int**] paragraph 1 as follows:

30.7.2 Specializations for integers [atomics.types.int]

1 There are specializations of the `atomic` class template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<cstdint>`. For each such type *`integral`*, the specialization `atomic<`*`integral`*`>` provides additional atomic operations appropriate to integral types. [*Note:* For the specialization `atomic<bool>`, see 30.7. — *end note*]

```
namespace std {
  template<> struct atomic<integral> {
    using value_type = integral;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(integral, memory_order = memory_order::seq_cst) volatile no
    void store(integral, memory_order = memory_order::seq_cst) noexcept;
    integral load(memory_order = memory_order::seq_cst) const volatile noe
    integral load(memory_order = memory_order::seq_cst) const noexcept;
    operator integral() const volatile noexcept;
    operator integral() const noexcept;
    integral exchange(integral, memory_order = memory_order::seq_cst) vola
    integral exchange(integral, memory_order = memory_order::seq_cst) noex
    bool compare_exchange_weak(integral&, integral,
                               memory_order, memory_order) volatile noexce
    bool compare_exchange_weak(integral&, integral,
                               memory_order, memory_order) noexcept;
    bool compare_exchange_strong(integral&, integral,
                                 memory_order, memory_order) volatile noex
    bool compare_exchange_strong(integral&, integral,
                                 memory_order, memory_order) noexcept;
    bool compare_exchange_weak(integral&, integral,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(integral&, integral,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(integral&, integral,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(integral&, integral,
                                 memory_order = memory_order::seq_cst) noe
```

```
    void wait(integral, memory_order = memory_order::seq_cst) const volati
    void wait(integral, memory_order = memory_order::seq_cst) const noexce
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
```

```
    integral fetch_add(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_add(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_sub(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_sub(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_and(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_and(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_or(integral, memory_order = memory_order::seq_cst) vola
    integral fetch_or(integral, memory_order = memory_order::seq_cst) noex
    integral fetch_xor(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_xor(integral, memory_order = memory_order::seq_cst) noe

    atomic() noexcept = default;
    constexpr atomic(integral) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    integral operator=(integral) volatile noexcept;
    integral operator=(integral) noexcept;

    integral operator++(int) volatile noexcept;
    integral operator++(int) noexcept;
    integral operator--(int) volatile noexcept;
    integral operator--(int) noexcept;
    integral operator++() volatile noexcept;
    integral operator++() noexcept;
    integral operator--() volatile noexcept;
    integral operator--() noexcept;
    integral operator+=(integral) volatile noexcept;
    integral operator+=(integral) noexcept;
    integral operator-=(integral) volatile noexcept;
    integral operator-=(integral) noexcept;
    integral operator&=(integral) volatile noexcept;
    integral operator&=(integral) noexcept;
    integral operator|=(integral) volatile noexcept;
    integral operator|=(integral) noexcept;
    integral operator^=(integral) volatile noexcept;
    integral operator^=(integral) noexcept;
  };
}
```

Modify [**atomics.types.float**] paragraph 1 as follows:

30.7.3 Specializations for floating-point types [atomics.types.float]

1 There are specializations of the `atomic` class template for the floating-point types `float`, `double`, and `long double`. For each such type *`floating-point`*, the specialization `atomic<`*`floating-point`*`>` provides additional atomic operations appropriate to floating-point types.

```
namespace std {
  template<> struct atomic<floating-point> {
    using value_type = floating-point;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(floating-point, memory_order = memory_order::seq_cst) volat
    void store(floating-point, memory_order = memory_order::seq_cst) noexc
    floating-point load(memory_order = memory_order::seq_cst) const volati
    floating-point load(memory_order = memory_order::seq_cst) const noexce
    operator floating-point() const volatile noexcept;
    operator floating-point() const noexcept;
    floating-point exchange(floating-point,
                            memory_order = memory_order::seq_cst) volatil
    floating-point exchange(floating-point,
                            memory_order = memory_order::seq_cst) noexcep
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order, memory_order) volatile noexce
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order, memory_order) noexcept;
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order, memory_order) volatile noex
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order, memory_order) noexcept;
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order = memory_order::seq_cst) noe

    void wait(floating-point, memory_order = memory_order::seq_cst) const
```

```
    void wait(floating-point, memory_order = memory_order::seq_cst) const
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;

    floating-point fetch_add(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_add(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_sub(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_sub(floating-point,
                                    memory_order = memory_order::seq_cst)

    atomic() noexcept = default;
    constexpr atomic(floating-point) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    floating-point operator=(floating-point) volatile noexcept;
    floating-point operator=(floating-point) noexcept;

    floating-point operator+=(floating-point) volatile noexcept;
    floating-point operator+=(floating-point) noexcept;
    floating-point operator-=(floating-point) volatile noexcept;
    floating-point operator-=(floating-point) noexcept;
  };
}
```

Modify [**atomics.types.pointer**] paragraph 1 as follows:

30.7.4 Partial specialization for pointers [atomics.types.pointer]

```
namespace std {
  template<class T> struct atomic<T*> {
    using value_type = T*;
    using difference_type = ptrdiff_t;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T*, memory_order = memory_order::seq_cst) volatile noexcept
    void store(T*, memory_order = memory_order::seq_cst) noexcept;
    T* load(memory_order = memory_order::seq_cst) const volatile noexcept;
    T* load(memory_order = memory_order::seq_cst) const noexcept;
    operator T*() const volatile noexcept;
    operator T*() const noexcept;
    T* exchange(T*, memory_order = memory_order::seq_cst) volatile noexcep
    T* exchange(T*, memory_order = memory_order::seq_cst) noexcept;
    bool compare_exchange_weak(T*&, T*, memory_order, memory_order) volati
    bool compare_exchange_weak(T*&, T*, memory_order, memory_order) noexce
    bool compare_exchange_strong(T*&, T*, memory_order, memory_order) noex
    bool compare_exchange_weak(T*&, T*,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(T*&, T*,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(T*&, T*,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(T*&, T*,
                                 memory_order = memory_order::seq_cst) noe
    void wait(T*, memory_order = memory_order::seq_cst) const volatile noe
    void wait(T*, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
```

```
    T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) volatile
    T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept
    T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) volatile
    T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept

    atomic() noexcept = default;
    constexpr atomic(T*) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T* operator=(T*) volatile noexcept;
    T* operator=(T*) noexcept;

    T* operator++(int) volatile noexcept;
    T* operator++(int) noexcept;
    T* operator--(int) volatile noexcept;
    T* operator--(int) noexcept;
    T* operator++() volatile noexcept;
    T* operator++() noexcept;
    T* operator--() volatile noexcept;
    T* operator--() noexcept;
    T* operator+=(ptrdiff_t) volatile noexcept;
    T* operator+=(ptrdiff_t) noexcept;
    T* operator-=(ptrdiff_t) volatile noexcept;
    T* operator-=(ptrdiff_t) noexcept;
  };
}
```

1 There is a partial specialization of the `atomic` class template for pointers. Specializations of this partial specialization are standard-layout structs. They each have a trivial default constructor and a trivial destructor.

Modify [**atomics.flag**] as follows:

30.9 Flag type and operations [atomics.flag]

```
namespace std {
  struct atomic_flag {
    bool test(memory_order = memory_order::seq_cst) const volatile noexcep
    bool test(memory_order = memory_order::seq_cst) const noexcept;
    bool test_and_set(memory_order = memory_order::seq_cst) volatile noexc
    bool test_and_set(memory_order = memory_order::seq_cst) noexcept;
    void clear(memory_order = memory_order::seq_cst) volatile noexcept;
    void clear(memory_order = memory_order::seq_cst) noexcept;
    void wait(bool, memory_order = memory_order::seq_cst) const volatile n
    void wait(bool, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;

    atomic_flag() noexcept = default;
    atomic_flag(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) volatile = delete;
  };

  bool atomic_flag_test(const volatile atomic_flag*) noexcept;
  bool atomic_flag_test(const atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(const volatile atomic_flag*, memory_order
  bool atomic_flag_test_explicit(const atomic_flag*, memory_order) noexcep
  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_orc
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexc
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noe
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
```

```
  void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
  void atomic_flag_wait(const atomic_flag*, bool) noexcept;
  void atomic_flag_wait_explicit(const volatile atomic_flag*, bool, memory
  void atomic_flag_wait_explicit(const atomic_flag*, bool, memory_order) n
  void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
  void atomic_flag_notify_one(atomic_flag*) noexcept;
  void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;
  void atomic_flag_notify_all(atomic_flag*) const noexcept;
```

```
  #define ATOMIC_FLAG_INIT see below
}
```

1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.

2 Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. — *end note*]

3 The `atomic_flag` type is a standard-layout struct. It has a trivial default constructor and a trivial destructor.

4 The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test(const volatile atomic_flag* object) noexcept;
bool atomic_flag_test(const atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(const volatile atomic_flag* object, memory_
bool atomic_flag_test_explicit(const atomic_flag* object, memory_order orc
bool atomic_flag::test(memory_order order = memory_order::seq_cst) const v
bool atomic_flag::test(memory_order order = memory_order::seq_cst) const n
```

5 *Expects:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.

6 *Effects:* Memory is affected according to the value of `order`, or according to `memory_order::seq_cst` for `atomic_flag_test`.

7 *Returns:* Atomically returns the value pointed to by `object` or `this`.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object,
                                       memory_order order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order o
bool atomic_flag::test_and_set(memory_order order = memory_order::seq_cst)
bool atomic_flag::test_and_set(memory_order order = memory_order::seq_cst)
```

8 *Effects:* Atomically sets the value pointed to by `object` or by `this` to `true`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (4.7).

9 *Returns:* Atomically, the value of the object immediately before the effects.

```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object,
                                memory_order order) noexcept;
void atomic_flag_clear_explicit(atomic_flag* object, memory_order order) n
void atomic_flag::clear(memory_order order = memory_order::seq_cst) volati
void atomic_flag::clear(memory_order order = memory_order::seq_cst) noexce
```

Expects: `order` is neither `memory_order::consume`, `memory_order::acquire`, nor `memory_order::acq_rel`.

Effects: Atomically sets the value pointed to by `object` or by `this` to `false`. Memory is affected according to the value of `order`.

```
void atomic_flag_wait(const volatile atomic_flag* object, bool old) noexce
void atomic_flag_wait(const atomic_flag* object, bool old) noexcept;
void atomic_flag_wait_explicit(const volatile atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag_wait_explicit(const atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag::wait(bool old,
                       memory_order order = memory_order::seq_cst) const v
void atomic_flag::wait(bool old,
                       memory_order order = memory_order::seq_cst) const n
```

10 *Expects:* `order` is neither `memory_order::release` nor `memory_order::acq_rel`.

11 *Effects:* Let `af` be `object` for the non-member functions and `this` for the member functions. Let `mo` be `memory_order::seq_cst` for `atomic_flag_wait` and the value of `order` for the other functions. Repeatedly performs the following steps, in order:

(11.1) — Evaluates `af->load(mo) != old`.

(11.2) — If the result of that evaluation is `true`, returns.

(11.3) — Blocks until it is unblocked by an atomic notifying operation or is unblocked spuriously.

12 *Remarks:* This function is an atomic waiting operation.

```
void atomic_flag_notify_one(volatile atomic_flag* object) noexcept;
void atomic_flag_notify_one(atomic_flag* object) noexcept;
void atomic_flag::notify_one() volatile noexcept;
void atomic_flag::notify_one() noexcept;
```

13 *Effects:* Given the set *W* of atomic waiting operations such that:

(13.1) — the atomic waiting operation blocked after observing the result of some atomic operation `X`,

(13.2) — `X` precedes some atomic operation `Y` in the modification order of `*object` or `*this`, and

(13.3) — `Y` happens before this call.

If the set *W* is not empty, unblocks the execution of one operation in set *W*.

14 *Remarks:* This function is an atomic notifying operation.

```
void atomic_flag_notify_all(volatile atomic_flag* object) const noexcept;
void atomic_flag_notify_all(atomic_flag* object) const noexcept;
void atomic_flag::notify_all() volatile noexcept;
void atomic_flag::notify_all() noexcept;
```

15 *Effects:* Unblocks the execution of all atomic waiting operations such that:

(15.1) — the atomic waiting operation blocked after observing the result of some atomic operation `X`,

(15.2) — `X` precedes some atomic operation `Y` in the modification order of `*object` or `*this`, and

(15.3) — `Y` happens before this call.

16 *Remarks:* This function is an atomic notifying operation.

Modify Table 134 "Thread support library summary" in **[thread.general]** as follows:

Table 134 — Thread support library summary

	Subclause	Header(s)
31.2	Requirements	
31.3	Threads	`<thread>`
31.4	Mutual exclusion	`<mutex>` `<shared_mutex>`
31.5	Condition variables	`<condition_variable>`
31.□	Semaphores	`<semaphore>`
31.□	Latches and barriers	`<latch>` `<barrier>`
31.6	Futures	`<future>`

Add two new subclauses after **[thread.condition]**:

31.□ Semaphores **[thread.semaphore]**

1 Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.

2 A counting semaphore is a semaphore object that models a non-negative resource count. A binary semaphore is a semaphore object that has only two states known as available and unavailable. [*Note:* A binary semaphore should be more efficient than a counting semaphore with a unit magnitude count. – *end note*]

31.□.1 Header `<semaphore>` synopsis **[thread.semaphore.syn]**

```
namespace std {
  template<ptrdiff_t least_max_value = implementation-defined>
    class counting_semaphore;

  using binary_semaphore = counting_semaphore<1>;
}
```

31.□.2 Class template `counting_semaphore` [thread.semaphore.counting.class]

```
namespace std {
  template<ptrdiff_t least_max_value>
  class counting_semaphore {
  public:
    static constexpr ptrdiff_t max() noexcept;

    constexpr explicit counting_semaphore(ptrdiff_t desired);
    ~counting_semaphore();

    counting_semaphore(const counting_semaphore&) = delete;
    counting_semaphore& operator=(const counting_semaphore&) = delete;

    void release(ptrdiff_t update = 1);
    void acquire();
    bool try_acquire() noexcept;
    template<class Rep, class Period>
      bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
    template<class Clock, class Duration>
      bool try_acquire_until(const chrono::time_point<Clock, Duration>& ab

  private:
    ptrdiff_t counter; // exposition only
  };
}
```

1 Class `counting_semaphore` maintains an internal counter that is initialized when the semaphore is created. The counter is decremented when a thread acquires the semaphore, and is incremented when a thread releases the semaphore. If a thread tries to acquire the semaphore when the counter is zero, the thread will block until another thread increments the counter by releasing the semaphore.

2 `least_max_value` shall be greater than zero.

3 `counting_semaphore`s permit concurrent invocation of the `release`, `acquire`, `try_acquire`, `try_acquire_for`, and `try_acquire_until` member functions.

```
static constexpr ptrdiff_t max() noexcept;
```

4 *Returns:* The maximum value of `counter`. This value is greater than or equal to `least_max_value`.

```
constexpr explicit counting_semaphore(ptrdiff_t desired);
```

5 *Expects:* `desired` is greater than or equal to zero and less than or equal to `max()`.

6 *Effects:* Initializes `counter` with `desired`.

7 *Throws:* Nothing.

```
~counting_semaphore();
```

8 *Expects:* For every function call blocked on `*this`, a function call that will cause it to unblock and return has happened before this call. [*Note:* This relaxes the usual rules, which would have required all blocking function calls to happen before destruction. — *end note*]

```
void release(ptrdiff_t update = 1);
```

9 *Expects:* `update` is greater than or equal to zero, and `counter + update` is less than or equal to `max()`.

10 *Effects:* Atomically execute `counter += update`. Then, unblock any threads that are waiting for `counter` to be greater than zero.

11 *Synchronization:* Strongly happens before invocations of `try_acquire` that observe the result of the effects.

12 *Throws:* `system_error` when an exception is required (31.2.2).

13 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

```
bool try_acquire() noexcept;
```

14 *Effects:*

(14.1) — With low probability, returns immediately. [*Note:* An implementation should ensure that `try_acquire` does not consistently return `false` in the absence of contending acquisitions. — *end note*]

(14.2) — Otherwise, if `counter` is greater than zero, atomically decrement `counter` by one.

15 *Returns:* `true` if `counter` was decremented, otherwise `false`.

```
void acquire();
```

16 *Effects:* Repeatedly performs the following steps, in order:

(16.1) — Evaluates `try_acquire`. If the result is `true`, returns.

(16.2) — Blocks on `*this` until `counter` is greater than zero.

17 *Throws:* `system_error` when an exception is required (31.2.2).

18 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

```
template<class Rep, class Period>
  bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
  bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_ti
```

19 *Effects:* Repeatedly performs the following steps, in order:

(19.1) — Evaluates `try_acquire`. If the result is `true`, returns `true`.

(19.2) — Blocks on `*this` until `counter` is greater than zero or until the timeout expires. If it is unblocked by the timeout expiring, returns `false`.

The timeout expires when the current time is after `abs_time` (for `try_acquire_until`) or when at least `rel_time` has passed from the start of the function (for `try_acquire_for`).

20 *Throws:* Timeout-related exceptions (31.2.4), or `system_error` when a non-timeout-related exception is required (31.2.2).

21 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

31.🔒 Coordination Types [thread.coord]

1 This subclause describes various concepts related to thread coordination, and defines the coordination types `latch` and `barrier`. These types facilitate concurrent computation performed by a number of threads. Concurrent invocations of the member functions of `latch` and `barrier`, other than their destructors, do not introduce data races.

31.□.1 Latches [thread.coord.latch]

1 A latch is a thread coordination mechanism that allows any number of threads to block until the latch is arrived at (via the `count_down` function) an expected number of times. The expected count is set when the latch is created. An individual latch is a single-use object; once the expected count has been reached, the latch cannot be reused.

31.□.1.1 Header `<latch>` synopsis [thread.coord.latch.syn]

```
namespace std {
  class latch;
}
```

31.□.1.2 Class `latch` [thread.coord.latch.class]

```
namespace std {
  class latch {
  public:
    constexpr explicit latch(ptrdiff_t expected);
    ~latch();

    latch(const latch&) = delete;
    latch& operator=(const latch&) = delete;

    void count_down(ptrdiff_t update = 1);
    bool try_wait() const noexcept;
    void wait() const;
    void arrive_and_wait(ptrdiff_t update = 1);

  private:
    ptrdiff_t counter; // exposition only
  };
}
```

1 A `latch` maintains an internal counter that is initialized when the `latch` is created. Threads can block on the latch object, waiting for `counter` to be decremented to zero.

```
constexpr explicit latch(ptrdiff_t expected);
```

2 *Expects:* `expected` is greater than or equal to zero.

3 *Effects:* Initializes `counter` with `expected`.

4 *Throws:* Nothing.

```
~latch();
```

5 *Expects:* No threads are blocked on `*this`. [*Note:* May be called even if some threads have not yet returned from invocations of `wait` on this object, provided that they are unblocked. This relaxes the usual rules, which would have required all blocking function calls to happen before destruction. - *end note*]

6 *Remarks:* The destructor may block until all threads have exited invocations of `wait` on this object.

```
void count_down(ptrdiff_t update = 1);
```

7 *Expects:* `update` is greater than or equal to zero, and `update` is less than or equal to `counter`.

8 *Effects:* Atomically decrements `counter` by `update`. If `counter` is equal to zero, unblocks all threads blocked on `*this`.

9 *Synchronization:* Synchronizes with the returns from all calls that are unblocked.

10 *Throws:* `system_error` when an exception is required (31.2.2).

11 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

```
bool try_wait() const noexcept;
```

12 *Returns:* `counter == 0`.

```
void wait() const;
```

13 *Effects:* If `counter` equals zero, returns immediately. Otherwise, blocks on `*this` until it is unblocked by a call to `count_down` that decrements `counter` to zero.

14 *Throws:* `system_error` when an exception is required (31.2.2).

15 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

```
void arrive_and_wait(ptrdiff_t update = 1);
```

16 *Effects:* Equivalent to:

(16.1) — `count_down(update);`

(16.2) — `wait();`

17 *Throws:* `system_error` when an exception is required (31.2.2).

18 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

31.□.2 Barriers [thread.coord.barrier]

1 A barrier is a thread coordination mechanism whose lifetime consists of a sequence of phases, where each phase allows a certain number of threads to arrive at the barrier and then wait for all the other threads to also arrive at the barrier. [*Note:* A barrier is useful for managing repeated tasks that are handled by multiple threads. - *end note*]

31.□.2.1 Header <barrier> synopsis **[thread.coord.barrier.syn]**

```
namespace std {
  template<class CompletionFunction = unspecified>
    class barrier;
}
```

31.□.2.2 Class template `barrier` [thread.coord.barrier.class]

```
namespace std {
  template<class CompletionFunction>
  class barrier {
  public:
    using arrival_token = see below;

    constexpr explicit barrier(ptrdiff_t expected,
                               CompletionFunction f = {});

    ~barrier();

    barrier(const barrier&) = delete;
    barrier& operator=(const barrier&) = delete;

    [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
    void wait(arrival_token&& arrival) const;

    void arrive_and_wait();
    void arrive_and_drop();

  private:
    CompletionFunction completion; // exposition only
  };
}
```

1 Each ***phase*** of the barrier consists of the following steps:

(1.1) — The expected count is reset to what was specified by the `expected` argument to the constructor, possibly adjusted by calls to `arrive_and_drop`.

(1.2) — The expected count is decremented by each call to `arrive`.

(1.3) — When the expected count reaches zero, the completion step is run on one of the threads that arrived at the barrier during the phase.

(1.4) — When the completion step finishes, the next phase starts.

2 Each phase defines a ***synchronization point***. Threads that arrive at the barrier during the phase can block on the phase's synchronization point by calling `wait`, and will remain blocked until the phase completes.

3 The ***completion step*** that is executed at the end of each phase has the following effects:

(3.1) — Invokes the completion function, equivalent to `completion()`.

(3.2) — Unblocks all threads that are blocked on the phase's synchronization point.

The end of the completion step strongly happens before the returns from all calls that were unblocked by the completion step.

4 `CompletionFunction` shall meet the *Cpp17MoveConstructible* (Table 26) requirements and *Cpp17Destructable* (Table 30) requirements. `is_invocable_r_v<void, CompletionFunction>` shall be `true`, and `noexcept(declval<CompletionFunction>()())` shall be `true`.

5 `barrier::arrival_token` is an unspecified type, such that `is_nothrow_move_constructible_v<barrier::arrival_token>` is `true` and `is_nothrow_move_assignable_v<barrier::arrival_token>` is `true`.

```
constexpr explicit barrier(ptrdiff_t expected,
                           CompletionFunction f = {});
```

6 *Expects:* `expected` is greater than or equal to zero.

7 *Effects:* Sets the initial expected count for each phase to `expected`. Initializes `completion` with `std::move(f)`. Starts the first phase. [*Note:* If `expected` is `0` this object can only be destroyed. — *end note*]

8 *Throws:* Any exception thrown by `CompletionFunction`'s move constructor.

```
~barrier();
```

9 *Expects:* No threads are blocked at a synchronization point for any phase of this object. [*Note:* May be called even if some threads have not yet returned from invocations of `wait`, provided that they have unblocked. This relaxes the usual rules, which would have required all blocking function calls to happen before destruction. - *end note*]

10 *Remarks:* The destructor may block until all threads have exited invocations of `wait` on this object.

```
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
```

11 *Expected:* `update` is greater than zero, and `update` is less than or equal to the expected count.

12 *Effects:* Constructs an object of type `arrival_token` that is associated with the `barrier`'s synchronization point for the current phase. Then, decrements the expected count by `update`.

13 *Synchronization:* The call to `arrive` strongly happens before the start of the completion step for the current phase.

14 *Returns:* The constructed `arrival_token` object.

15 *Throws:* `system_error` when an exception is required (31.2.2).

16 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

17 *Remarks:* This call can cause the completion step for the current phase to start.

```
void wait(arrival_token&& arrival) const;
```

18 *Expects:* `arrival` is associated with the synchronization point for the current or the immediately preceding phase of the same barrier object.

19 *Effects:* Blocks at the synchronization point associated with `std::move(arrival)` until the completion step of the synchronization point's phase is run. [*Note:* If `arrival` is associated with the synchronization point for a previous phase, the call returns immediately. - *end note*]

20 *Throws:* `system_error` when an exception is required (31.2.2).

21 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

```
void arrive_and_wait();
```

22 *Expects:* The expected count for the current phase is greater than zero.

23 *Effects:* Equivalent to `wait(arrive())`.

24 *Throws:* `system_error` when an exception is required (31.2.2).

25 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

26 *Remarks:* This call can cause the completion step for the current phase to start.

```
void arrive_and_drop();
```

27 *Expects:* The expected count for the current phase is greater than zero.

28 *Effects:* Decrements the initial expected count for all subsequent phases by one. Then decrements the expected count for the current phase by one.

29 *Synchronization:* The call to `arrive_and_drop` strongly happens before the start of the completion step for the current phase.

30 *Throws:* `system_error` when an exception is required (31.2.2).

31 *Error conditions:* Any of the error conditions allowed for mutex types (31.4.3.2).

32 *Remarks:* This call can cause the completion step for the current phase to start.

Create the following feature test macros with the given headers, adding them to the table in [support.limits.general]:

— `__cpp_lib_atomic_lock_free_type_aliases` in `<atomic>`, which implies that `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types are available.

— `__cpp_lib_atomic_flag_test` in `<atomic>`, which implies the `test` methods and free functions for `atomic_flag` are available.

— `__cpp_lib_atomic_wait` in `<atomic>`, which implies the `notify_*` and `wait` methods and free functions for `atomic` and `atomic_flag` are available.

— `__cpp_lib_semaphore` in `<semaphore>`, which implies that `counting_semaphore` and `binary_semaphore` are available.

— `__cpp_lib_latch` in `<latch>`, which implies that `latch` is available.

— `__cpp_lib_barrier` in `<barrier>`, which implies that `barrier` is available.

§ Index

§ Terms defined by this specification

§ References

§ Informative References

[N4791]

Richard Smith. Working Draft, Standard for Programming Language C++. 26 November 2018. URL: https://wg21.link/n4791

[P0514R4]

Olivier Giroux. Efficient concurrent waiting for C++20. 3 May 2018. URL: https://wg21.link/p0514r4

[P0666R2]

Olivier Giroux. Revised Latches and Barriers for C++20. 6 May 2018. URL: https://wg21.link/p0666r2

[P0995R1]

JF Bastien, Olivier Giroux, Andrew Hunter. Improving atomic_flag. 22 June 2018. URL: https://wg21.link/p0995r1

[P1258R0]

Detlef Vollmann. Don't Make C++ Unimplementable On Small CPUs. 8 October 2018. URL: https://wg21.link/p1258r0